Введение

Практикум курса — девять лабораторных работ на собственном стенде студента. За работы 1–8 студент собирает платформу данных вокруг реального открытого источника: сборщик данных, озеро данных, таблицы lakehouse, аналитическое хранилище с витриной и дашбордом, поток событий через брокер, пакетный конвейер по расписанию и потоковую обработку. Работа 9 необязательна: в ней один вопрос к данным решается всеми тремя архитектурами хранения и они сравниваются.

Работы образуют одну цепочку: каждая опирается на результат предыдущей и на данные, накопленные сборщиком с первой недели. Порядок выполнения обязателен.

Практикум выдаётся двумя документами:

- **методические указания** — этот документ. Общий для потока справочник на семестр: порядок работ, оценивание и защита, требования к стенду и его запуск, источники данных и форма паспорта источника, контракт сборщика, справочник по сервисам стенда, форма каталога данных. Заданий он не содержит;
- **задание** — отдельный документ со своим вариантом: параметры варианта и девять работ. Задание ссылается на разделы методических указаний по названию.

Файлы стенда — файлы запуска сервисов, оболочка службы сборщика и образец файла переменных варианта — лежат в открытом репозитории стенда: адрес и первый запуск — в разделе «Стенд курса». Писать их самому не нужно.

Порядок работ

- Работы 1–8 обязательны. Каждая делится на основное задание (обязательно, нужно следующей работе) и дополнительное (необязательно, даёт баллы сверх основных).
- Работа N принимается только после приёмки основного задания работы N–1. Работа N+1 не опирается на результаты дополнительного задания работы N.
- Работа 9 необязательна целиком: основного задания у неё нет, она не блокирует зачёт.
- Защищать работу N можно не раньше лекции N; выполнять — заранее.
- Шаг с пометкой «по готовому файлу» выполняется файлом репозитория стенда, шаг с пометкой «по инструкции» — командами справочного раздела методических указаний. Понятия, на которых они держатся, раскрываются в следующих лекциях; на защите такой шаг объясняется на уровне того, что делает команда.
- Работа 1 защищается не позднее лабораторного занятия второй недели: работы 3–9 опираются на историю, которую сборщик накапливает с начала записи, а работы 5, 7 и 9 требуют её глубины — трёх, четырёх и шести недель. При более поздней защите недостающие недели добираются историческим срезом источника (раздел «Источники данных»); если история источника для этого не годится, преподаватель заменяет источник варианта.
- Работы N и N+1 можно защищать в одну защиту, если после защит одной работы остаётся время занятия: основное задание работы N принимается первым. Иначе сдвоенная защита переносится на следующее занятие.

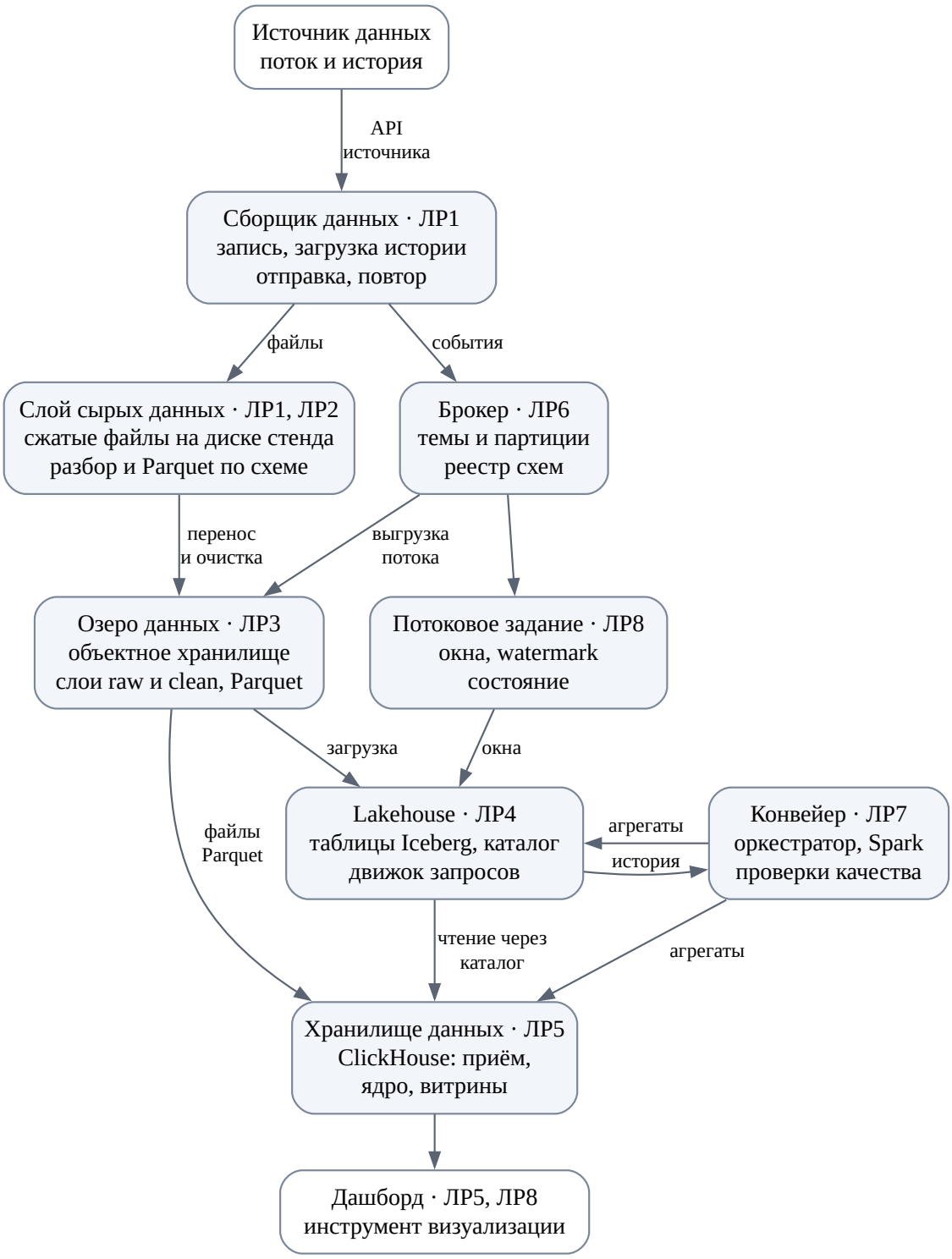


Рисунок 1.1: Платформа данных практикума: звенья и работы, которые их строят.

Баллы и сроки

Шкала стобалльная. Основные задания работ 1–8 дают 100 баллов, дополнительные задания — до 12, работа 9 — 12. Сумма выше 100 усекается до 100. Баллы не округляются: порог сравнивается с дробной суммой.

Работа принимается целиком: баллы начисляются, когда выполнены все пункты приёмки и даны ответы на вопросы защиты (раздел «Защита»). Работа с невыполненным пунктом или без ответов не принимается и защищается повторно. Так же целиком принимаются дополнительное задание и работа 9.

Контрольная точка	Основное	Дополнительное	Занятие
ЛР1 Стенд и сбор данных	8	1	1
ЛР2 Форматы данных	10	1	2
ЛР3 Озеро данных	12	1	3
ЛР4 Таблицы Iceberg	15	2	4
ЛР5 Хранилище и витрина	15	2	5
ЛР6 Поток событий	15	2	6
ЛР7 Пакетная обработка и конвейер	15	2	7
ЛР8 Потоковая обработка	10	1	8
ЛР9 Исследование данных (необязательная)	—	12	9

Таблица 1.1: Контрольные точки и их вес.

Коэффициент за срок считается от занятия контрольной точки по занятию, на котором работа принята:

- работы 1–7: на занятии своего номера — 100 %, на следующем — 80 %, на любом более позднем до девятого включительно — 60 %;
- работа 8: восьмое занятие — 100 %, девятое — 80 %;
- работа 9: только девятое занятие — 100 %;
- дополнительное задание работ 1–8 защищается только в ту же защиту, что и основное, и с тем же коэффициентом.

Девятое занятие — последнее: после него работы не принимаются.

Пункт основного задания, который не выполняется по причине, не зависящей от студента (сервис варианта не поднимается на его машине, операция не поддержана продуктом варианта, источник отказал), называется преподавателю до защиты. Такое задание может быть принято с зафиксированным дефектом за половину баллов — не больше одной работы за семестр.

Условие зачёта: не менее 60 баллов и защищены основные задания работ 1–8. Суммы основных заданий хватает на зачёт и при защите всех работ на девятом занятии: $90 \times 0{,}6 + 10 \times 0{,}8 = 62$.

Защита

Приёмка идёт целиком на защите: отдельной сдачи файлов нет, стенд студент приносит на своей рабочей машине. Раздел «Приёмка» каждой работы перечисляет результаты, которые предъявляются на защите. Преподаватель может попросить выполнить на стенде любой шаг или открыть любой файл.

На защите студент:

- объясняет принятые решения и по требованию повторяет любой шаг;
- отвечает на вопросы по понятиям лекции того же номера, на защите работы 9 — лекций 3–5.

Без объяснения решений и ответов на вопросы работа не принимается и защищается повторно; коэффициент за срок считается по занятию повторной защиты.

Всё, что предъявляется на защите, живёт в рабочем репозитории на стенде (раздел «Стенд курса»): программа сборщика, схемы, задания и конвейеры, паспорт источника `SOURCE.md`, каталог данных `CATALOG.md` и журнал работ `NOTES.md` с решениями и замерами по каждой работе.

Правила практикума

- **Данные — собственные.** Работа выполняется на данных, собранных своим сборщиком из источника своего варианта. Работа на данных, полученных от другого студента, не принимается; копирование чужого рабочего репозитория или собранных данных — нарушение для обеих сторон.
- **Рабочий репозиторий не публикуется.** Удалённая копия рабочего репозитория допускается только закрытой: открытая копия с решениями работ считается передачей решений.
- **Пропуски восполняются историей.** Пропуск в записи потока из-за выключенной рабочей машины восполняется историей источника за период пропуска (раздел «Источники данных», строка «История»). У Wikimedia, GH Archive, GDELT, sensor.community и adsb.lol история приводится к схеме события и закрывает пропуск теми же событиями. У MOEX ISS история — минутные свечи: они ложатся отдельным набором, а пропуск в наборе событий объясняется в журнале работ. Работа не отклоняется из-за пропусков, если они объяснены и восполнены.
- **Свой источник** принимается по согласованию с преподавателем, если он проходит требования паспорта источника: лицензия допускает учебное использование, есть исторический срез и поток или его воспроизведение. Остальные оси варианта при этом не меняются.
- **Замена источника.** Источник, ставший недоступным или изменивший условия использования, заменяется решением преподавателя: студент получает резервный источник, собранные ранее данные остаются частью сдаваемого результата, приёмка уже защищённых работ не пересматривается, остальные оси варианта не меняются. Паспорт источника заводится заново.

ИИ-ассистенты и агенты

ИИ-ассистенты и агенты разрешены без ограничений. Автор решения — студент: неспособность объяснить сданное — нарушение, работа не принимается (раздел «Защита»). Правила безопасной работы с агентом:

- сначала задание и план, затем изменения: агент получает задачу с критерием готовности и предлагает план до правки файлов;
- каждая внесённая правка читается до того, как принята;
- порождённый код и запросы проверяются на собственных данных, а не на вид: типичные ошибки агента — выдуманные поля схемы, неверная единица времени, запрос, считающий не то, что спрошено;
- ключи источников и учётные данные стенда не попадают в контекст агента: файл `.env` закрывается от агента правилами доступа инструмента, агенту показывается `.env.example`; ключ, показанный агенту, считается утёкшим и заменяется;
- у агента нет доступа к чужим стендам, к боевым системам и к публикации собранных данных.

Лицензии данных

Данные внешнего источника используются по его лицензии и условиям. Лицензия, условия использования, объявленные лимиты обращений и требования к представлению клиента записываются в паспорт источника и соблюдаются сборщиком. Сведения по источникам курса — раздел «Источники данных». Собранные данные остаются на стенде студента и не публикуются.

Литература

- Клеппман М. Высоконагруженные приложения: программирование, масштабирование, поддержка. — Спринт Бук, 2026.

- Петров А. Распределённые данные: алгоритмы работы современных систем хранения информации. — Спринт Бук, 2025.
- Шапира Г. и др. Apache Kafka: потоковая обработка и анализ данных. 2-е изд. — Питер, 2023.
- Перрен Ж.-Ж. Spark в действии. 2-е изд. — ДМК Пресс, 2021.
- Уилке К. Основы визуализации данных. — Бомбора, 2024.
- Shiran T., Hughes J., Merced A. Apache Iceberg: The Definitive Guide. — O'Reilly, 2024.
- Kimball R., Ross M. The Data Warehouse Toolkit. 3rd ed. — Wiley, 2013.

Документация: Apache Iceberg, Apache Parquet, Apache Kafka, Apache Spark, Apache Flink, Apache Airflow, ClickHouse, Trino, DuckDB.

Инженерные публикации российских компаний о платформах данных: блоги VK Tech, Avito, X5 Tech, МТС и Lamoda на Хабре.

Стенд курса

Стенд — одна виртуальная машина на рабочей машине студента, на Windows — дистрибутив WSL2. Сервисы стенда работают в контейнерах и запускаются профилями: профиль работы N поднимает сервисы её задания и постоянно работающие. Работы обращаются к сервисам по именам ролей и портам, поэтому способ создания машины на задания не влияет.

Требования к стенду

Параметр	Стенд	Лёгкий вариант
Процессоры	4 vCPU	4 vCPU
Память стенда	12 ГБ	6 ГБ
Диск стенда	100 ГБ	100 ГБ
Память рабочей машины	16 ГБ	8 ГБ
Система стенда	Ubuntu Server 24.04 LTS, amd64 или arm64	та же
Docker Compose	2.20 и новее	та же

Таблица 2.1: Размеры стенда; значения минимальные.

Размеры получены прогоном профилей самого тяжёлого варианта (StarRocks, DataLens, Kafka с реестром схем, Airflow, Flink) под нагрузкой на машине 12 ГБ и 4 vCPU без подкачки: пик профиля — 6,5 ГиБ, запаса хватает на задание Spark и кэш системы. Диск делится так: система — около 5 ГБ, образы сервисов — до 30 ГБ, слой сырых данных — до 20 ГБ (предел из контракта сборщика), остальное — копия сырья в озере, очищенный слой, таблицы, витрина и журнал брокера. У самых объёмных фильтров (доски TQBR и RFUD, круг Франкфурта) эти слои за восемь недель занимают около 35 ГБ; сроки хранения слоёв назначает работа 3.

Рабочая машина с 16 ГБ памяти — условие допуска к курсу. Студент, чья машина меньше, обращается к преподавателю до защиты работы 1 и получает номер лёгкого варианта: его пик памяти — около 2,1 ГиБ. Номер меняется только до защиты работы 1: он задаёт источник и фильтр потока, и собранные данные должны им отвечать.

Архитектура стенда совпадает с архитектурой рабочей машины: на Apple Silicon — arm64, на остальных — amd64. Все образы стенда собираются под обе архитектуры.

Порты

Порты публикуются на стенде. На WSL2 они доступны с рабочей машины как `localhost`, у виртуальной машины — по её адресу. Изнутри стенда сервисы обращаются друг к другу по именам ролей. Клиенты брокера запускаются на стенде: брокер сообщает клиенту адрес `localhost`, и с рабочей машины подключение работает только на WSL2.

Роль	Порт	Переменная	С работы
`collector` — счётчики сборщика	9101	`PORT_COLLECTOR`	ЛР1

`s3` — объектное хранилище, API	9000	`PORT_S3`	ЛР3
`s3` — интерфейс SeaweedFS, у Garage — административный API	9001	`PORT_S3_UI`	ЛР3
`catalog` — каталог таблиц	8181	`PORT_CATALOG`	ЛР4
`engine` — интерфейс Trino и StarRocks, SQL Trino	8080	`PORT_ENGINE`	ЛР4
`engine` — SQL StarRocks	9030	`PORT_ENGINE_SQL`	ЛР4
`warehouse` — ClickHouse, HTTP	8123	`PORT_WAREHOUSE_HTTP`	ЛР5
`warehouse` — ClickHouse, родной протокол	9002	`PORT_WAREHOUSE_NATIVE`	ЛР5
`bi` — инструмент визуализации	8088	`PORT_BI`	ЛР5
`broker` — брокер	9092	`PORT_BROKER`	ЛР6
`registry` — реестр схем	8081	`PORT_REGISTRY`	ЛР6
`orchestrator` — оркестратор	8082	`PORT_ORCHESTRATOR`	ЛР7
`stream` — интерфейс потокового задания	8083	`PORT_STREAM`	ЛР8
`spark` — интерфейс пакетного задания, пока оно идёт	4040	`PORT_SPARK_UI`	ЛР4

Таблица 2.2: Порты стенда.

Порт по умолчанию меняется только тогда, когда он занят на рабочей машине: значение переменной задаётся в `.env`, остальные порты не трогаются. У встраиваемого движка DuckDB портов нет: он запускается командой.

Развёртывание

Виртуальная машина

Подходит любой гипервизор: VirtualBox, VMware Workstation или Fusion, Hyper-V, libvirt/QEMU, UTM, Parallels. На Windows вместо виртуальной машины используется WSL2 (следующий подраздел).

1. Создайте виртуальную машину с параметрами из таблицы требований и установите Ubuntu Server 24.04 LTS. Пользователь, созданный при установке, получает uid 1000: под ним работают сборщик и встраиваемый движок, и каталоги данных стенда отдаются ему.
2. Подключите машину к сети NAT гипервизора, в которой она видна рабочей машине: Default Switch у Hyper-V, Shared у UTM и Parallels, NAT у VMware и libvirt; у VirtualBox — NAT и второй адаптер «только узел». Режим «сетевой мост» не используется: сервисы стенда работают без паролей, и в этом режиме их порты видны всей сети.
3. Включите вход по SSH с рабочей машины: работа со стендом идёт через терминал и браузер рабочей машины.

WSL2 на Windows

1. Задайте ресурсы WSL2 в файле `%UserProfile%\.wslconfig` и отключите остановку дистрибутива без открытых сеансов — иначе служба сборщика остановится вместе с последним терминалом:

```
[wsl2]
memory=12GB
processors=4

[general]
instanceIdleTimeout=-1
```

2. Установите дистрибутив и проверьте, что в нём работает systemd: без него службы стенда не запускаются сами. Ответ `running` или `degraded` означает, что systemd работает. Иначе в `/etc/wsl.conf` дистрибутива добавляется раздел `[boot]` со строкой `systemd=true`, затем выполняется `wsl --shutdown`.

```
wsl --install -d Ubuntu-24.04
wsl -d Ubuntu-24.04 -- systemctl is-system-running
```

3. Файлы стенда кладутся в файловую систему дистрибутива (`~/stand`), а не на диск Windows (`/mnt/c`): там права доступа и скорость ввода-вывода Linux.
4. После перезагрузки Windows дистрибутив запускается вместе с первым терминалом. Чтобы сборщик продолжал запись без входа в терминал, заведите задачу планировщика Windows при входе в систему с командой `wsl.exe -d Ubuntu-24.04 --exec /bin/true`.

Docker Engine

На стенде ставится Docker Engine с плагином Compose — пакетами Ubuntu или из репозитория Docker. Docker Desktop не нужен.

```
sudo apt-get update
sudo apt-get install -y docker.io docker-compose-v2 git python3-venv jq
sudo usermod -aG docker $USER        # затем перелогиниться
sudo systemctl enable --now docker
docker compose version
```

Если загрузка образов с Docker Hub отвечает ошибкой 429 или не проходит, в `/etc/docker/daemon.json` задаётся зеркало реестра, после чего движок перезапускается (`sudo systemctl restart docker`):

```
{ "registry-mirrors": ["https://mirror.gcr.io"] }
```

Файлы стенда и первый запуск

Файлы стенда лежат в открытом репозитории `https://github.com/nitwof/bigdata-stand` и клонируются в домашний каталог пользователя стенда. Первый запуск:

```
git clone https://github.com/nitwof/bigdata-stand ~/stand
cd ~/stand
git config user.name "Фамилия Имя"      # автор коммитов
git config user.email <адрес>
cp .env.example .env                    # параметры варианта — задание, ЛР1
docker compose run --rm stand-init      # каталоги данных, один раз
docker compose build collector          # оболочка службы сборщика
```

Службу сборщика поднимает шаг 8 работы 1, когда программа сборщика написана: без `collector/src/main.py` оболочка службы завершается с кодом 78 и перезапускается.

Каталог `data/` создаёт одноразовый сервис `stand-init` и раздаёт подкаталоги сервисам, работающим от разных пользователей. Остальное стенд готовит сам при первом запуске нужного профиля: bucket озера — профиль работы 3, склад каталога таблиц — профиль работы 4. Эти шаги повторяются при каждом запуске профиля и ничего не портят. Образы сервисов, которые собираются из файлов стенда, собираются первым запуском профиля: сеть нужна только в этот момент.

Вариант складывается из базового файла `compose.yaml` и файла на каждую зафиксированную ось. Перечень файлов — переменная `COMPOSE_FILE` в `.env`: в работе 1 в ней только `compose.yaml`, каждая следующая работа добавляет файл своей оси (задание называет его).

Рабочий репозиторий

Клон `~/stand` — рабочий репозиторий студента. Рядом с готовыми файлами стенда в нём живёт всё, что студент пишет сам и предъявляет на защите:

Путь	Что там	С работы
`collector/src/`	программа сборщика, точка входа `main.py`	ЛР1
`collector/requirements.txt`	зависимости сборщика	ЛР1
`SOURCE.md`	паспорт источника	ЛР1
`CATALOG.md`	каталог данных платформы	ЛР1
`NOTES.md`	журнал работ: решения, замеры, ответы на вопросы заданий — по разделу на работу	ЛР1
`schemas/`	схема события	ЛР2
`jobs/`	запросы и задания движков: очистка, загрузка, пакетные и потоковые задания	ЛР2
`conf/trino/`, `conf/spark/`	подключения движка Trino и пакетного движка Spark к каталогу таблиц и хранилищу (раздел «Сервисы стенда»)	ЛР4
`dags/`	конвейер оркестратора	ЛР7

Таблица 2.3: Рабочий репозиторий.

Каталог `data/` и файл `.env` в репозиторий не входят (`.gitignore`): первый — рабочие данные, второй — учётные данные стенда. Вспомогательные программы (проверка схемы, потребитель брокера) запускаются на стенде в виртуальном окружении Python (`python3 -m venv .venv`) и обращаются к сервисам по `localhost` и портам из таблицы; программы в контейнерах — по именам ролей.

Исправления стенда

Готовые файлы стенда — файлы `compose*.yaml`, каталог `services/`, оболочка службы сборщика, `.env.example` — не правятся. Исправление стенда в течение семестра публикуется в репозитории стенда и забирается в рабочий репозиторий:

```
1  git pull --no-rebase --no-edit                # слияние с исправлением
2  docker compose --profile labN up -d --build   # N — текущая работа
```

Изменённый по ошибке готовый файл возвращается командой `git checkout -- <файл>`. Файл `.env` исправление не меняет: новые переменные получают значения по умолчанию в файлах запуска.

Профили работ

Профиль работы поднимает сервисы её задания поверх постоянно работающих: сборщика, с работы 3 — объектного хранилища, с работы 4 — каталога таблиц. Весь стек одновременно не запускается ни на одной работе. Наборы данных предыдущих работ хранятся на диске стенда и читаются после запуска своих сервисов.

Работа	Профиль поднимает	Останавливается при переходе
ЛР1	`collector`	—
ЛР2	`collector`; `duckdb` по требованию	—
ЛР3	+ `s3`	—
ЛР4	+ `catalog`, `engine`; `spark` по требованию	—
ЛР5	+ `warehouse`, `bi`	—
ЛР6	`collector`, `s3`, `catalog`, `broker`, `registry`	`engine`, `warehouse`, `bi`
ЛР7	`collector`, `s3`, `catalog`, `engine`, `warehouse`, `orchestrator`	`broker`, `registry`
ЛР8	`collector`, `s3`, `catalog`, `warehouse`, `bi`, `broker`, `registry`, `stream`	`engine`, `orchestrator`
ЛР9	`collector`, `s3`, `catalog`, `engine`, `warehouse`, `bi`	`broker`, `registry`, `stream`

Таблица 2.4: Профили работ.

Переход к следующей работе — остановка профиля предыдущей и запуск профиля новой: остаётся ровно то, что нужно новой работе. Сборщик при этом перезапускается и продолжает запись с места остановки (раздел «Контракт сборщика данных»).

```
docker compose --profile lab5 stop
docker compose --profile lab6 up -d
docker compose ps                        # что работает сейчас
```

Сервисы `spark`, `duckdb` и у варианта Spark Structured Streaming потоковое задание `stream` постоянно не работают: они запускаются командой `docker compose run` на время задания (раздел «Сервисы стенда»). Полная остановка стенда перечисляет все профили:

```
docker compose --profile lab1 --profile lab2 --profile lab3 --profile lab4 \
  --profile lab5 --profile lab6 --profile lab7 --profile lab8 --profile lab9 \
  --profile tools down
```

Точка восстановления и копия сырых данных

Восемь недель собранных данных невосполнимы: глубины истории источника хватает не на любой пропуск. Поэтому у стенда две страховки.

Точка восстановления после работы 1 возвращает стенд в состояние после защиты работы 1. Создаётся одним из способов и называется датой создания:

- снимок виртуальной машины средствами гипервизора;
- экспорт дистрибутива WSL2 в каталог на диске Windows; в примерах это `D:\bigdata`, из WSL2 он виден как `/mnt/d/bigdata`:

```
New-Item -ItemType Directory -Force "D:\bigdata"
wsl --terminate Ubuntu-24.04
wsl --export Ubuntu-24.04 "D:\bigdata\stand-lr1-$(Get-Date -Format yyyy-MM-dd).tar"
```

- архив каталога стенда вместе с данными; каталоги данных принадлежат разным пользователям, поэтому архив создаётся с правами администратора:

```
docker compose --profile lab1 stop
sudo tar czf ~/stand-lr1-$(date +%F).tgz -C ~ stand
docker compose --profile lab1 up -d
```

Архив затем копируется на рабочую машину: внутри стенда он не защищает от потери стенда.

Копия слоя сырых данных лежит вне виртуальной машины, датирована и обновляется не реже раза в неделю. Файлы слоя уже сжаты, поэтому архив собирается без сжатия:

```
# виртуальная машина: команда выполняется на рабочей машине
ssh <пользователь>@<адрес стенда> "tar cf - -C stand/data raw" > raw-$(date +%F).tar
# WSL2: команда выполняется на стенде, копия уходит на диск Windows
tar cf /mnt/d/bigdata/raw-$(date +%F).tar -C ~/stand/data raw
```

Обе страховки проверяются на защите работы 3.

Проверка без веб-интерфейса

У части значений осей веб-интерфейса нет: у объектного хранилища Garage и у встраиваемого движка DuckDB. Их состояние проверяется из командной строки; команды подключения — раздел «Сервисы стенда».

Что проверить	Команда
bucket и объекты Garage	`docker compose exec s3 /garage bucket info lake`
bucket SeaweedFS без интерфейса	`weed shell` — подраздел «Объектное хранилище» раздела «Сервисы стенда»
объекты по префиксу в любом хранилище	DuckDB: `SELECT * FROM glob('s3://lake/raw/**') LIMIT 10`
таблицы Iceberg	DuckDB после `ATTACH`: `SHOW ALL TABLES`
что запущено и как завершилось	`docker compose ps -a`, `docker compose logs <роль>`

Таблица 2.5: Проверки из командной строки.

Источники данных

У каждого источника курса есть поток (живой или новые файлы по расписанию) и исторический срез, лицензия допускает учебное использование, ключ доступа не нужен. Задание назначает фильтр потока — подмножество, которое собирает сборщик: проекты, страны, регион, доску или типы событий. Фильтр держит восемь недель потока в пределе объёма слоя сырых данных — 20 ГБ сжатых файлов.

Строка «История» у каждого источника называет, чем закрывается пропуск в записи потока: теми же событиями или отдельным набором со своей схемой.

Объёмы ниже измерены перед семестром на коротком интервале и служат ориентиром: доли потока меняются в течение суток и недели. Паспорт источника студента приводит объёмы, измеренные на его собственных данных.

Общие правила обращения ко всем источникам:

- каждый запрос несёт заголовок `User-Agent` с названием программы и контактом (адрес почты);
- частота обращений не выше объявленной источником, а при её отсутствии — не выше указанной здесь;
- ответы 429 и 5xx обрабатываются повтором с растущей паузой, а не немедленным повтором;
- время события приводится к UTC; если источник отдаёт местное время, в паспорте указывается его часовой пояс.

MOEX ISS

Параметр	Значение
Поток	опрос `https://iss.moex.com` по пути `/iss/engines/<e>/markets/<m>/boards/<b>/securities.json`, где `<e>/<m>/<b>` — площадка, рынок и доска из фильтра задания; таблица `marketdata`: последняя цена, объёмы, время обновления; без авторизации данные идут с задержкой 15 минут
История	минутные свечи бумаги: `/iss/engines/<e>/markets/<m>/boards/<b>/securities/<SECID>/candles.json` с параметрами `interval=1`, `from` и `till`; колонки `open`, `close`, `high`, `low`, `value`, `volume`, `begin`, `end`, время московское; до 500 строк на страницу (параметр `start`). Запрос идёт по одной бумаге: семь торговых дней доски TQBR — около 2800 запросов. Свечи — отдельный набор со своей схемой, к событию они не приводятся
Лицензия и условия	информация ISS предоставляется для ознакомительного использования, иное использование — по договору с биржей; собранные данные не публикуются
Лимиты	не объявлены; опрос доски не чаще раза в 10 с, параметры `iss.meta=off` и `iss.only=marketdata` сокращают ответ; загрузка свечей — не чаще запроса в секунду
Событие	обновление строки `marketdata` бумаги; ключ — код бумаги `SECID` и время обновления `SYSTIME`; время события — `SYSTIME`

Таблица 3.1: MOEX ISS.

Известные дефекты: время — московское, без часового пояса; вне торговой сессии, в выходные и праздники поток пуст; строка повторяется в снимках без изменений; часть полей пуста у неторгуемых бумаг.

Фильтр задания (`SOURCE_FILTER`)	Бумаг в снимке	ГБ в торговый день сжатых	ГБ за 8 недель
`stock/shares/TQBR` — акции	506	0,34	13,5
`stock/bonds/TQOB` — ОФЗ	62	0,08	3,2
`currency/selt/CETS` — валюта	200	0,10	4,0
`futures/forts/RFUD` — фьючерсы	501	0,27	11

Таблица 3.2: MOEX ISS: фильтры и объёмы при опросе раз в 10 с; сжатие JSON ×5,2; восемь недель — 40 торговых дней.

Весь рынок одной доской не опрашивается: фильтр задания и есть доска.

Wikimedia EventStreams

Параметр	Значение
Поток	`https://stream.wikimedia.org/v2/stream/recentchange` — поток правок всех проектов в формате SSE; фильтр по полю `wiki` применяется на стороне сборщика
История	параметр `since` того же потока — от 7 до 31 суток назад: те же события и ключ, читается весь поток, фильтр — на стороне сборщика
Лицензия и условия	условия использования Фонда Викимедиа; тексты проектов — CC BY-SA 4.0, Викиданные — CC0
Лимиты	заголовок `User-Agent` с контактом обязателен; одно подключение к потоку; запросы к API проекта — последовательно, с параметром `maxlag`
Событие	правка или запись журнала; ключ — `meta.id`; время события — `meta.dt` (поле `timestamp` — секунды Unix)

Таблица 3.3: Wikimedia EventStreams.

Известные дефекты: служебные события с `meta.domain` = `canary`; повтор событий после переподключения по `Last-Event-ID`; у записей журнала нет полей размера страницы.

Фильтр задания (`SOURCE_FILTER`)	Событий в минуту	ГБ в сутки сжатых	ГБ за 8 недель
`enwiki`	240	0,023	1,3
`wikidatawiki`	420	0,040	2,2
`commonswiki`	770	0,073	4,1
`ruwiki,ukwiki,bewiki`	33	0,003	0,2
`dewiki,frwiki,eswiki,itwiki`	100	0,010	0,5

Таблица 3.4: Wikimedia: фильтры и объёмы; сжатие JSONL ×4,9.

Весь поток без фильтра — 2330 событий в минуту, 0,22 ГБ в сутки, 12,4 ГБ за восемь недель.

GH Archive

Параметр	Значение
Поток	почасовые архивы `https://data.gharchive.org/<ГГГГ-ММ-ДД>-<час>.json.gz` (час без ведущего нуля): новый файл появляется после окончания часа; поток — загрузка нового часа, воспроизведение — режим повтора сборщика
История	те же файлы, сплошь с 12 февраля 2011 года: те же события и ключ
Лицензия и условия	код проекта — MIT, материалы сайта — CC BY 4.0; события — публичные данные GitHub, условия — GitHub Terms of Service
Лимиты	не объявлены; файлы загружаются по одному
Событие	событие GitHub; ключ — `id`; время события — `created_at` (UTC); фильтр — по полю `type`

Таблица 3.5: GH Archive.

Известные дефекты: часть часовых файлов неполна (размер в разы меньше соседних); файл часа появляется с задержкой; 90 % событий — `PushEvent`.

Фильтр задания (`SOURCE_FILTER`)	Событий в минуту	ГБ в сутки сжатых	ГБ за 8 недель
`CreateEvent`, `DeleteEvent`, `ReleaseEvent`	180	0,034	1,9
`PullRequestEvent`, `IssuesEvent`, `IssueCommentEvent`	24	0,034	1,9
`PullRequestEvent`, `PullRequestReviewEvent`, `PullRequestReviewCommentEvent`	17	0,011	0,6

Таблица 3.6: GH Archive: фильтры и объёмы; сжатие JSONL после фильтра ×4,7–6,9.

Весь поток без фильтра — 2160 событий в минуту, 0,41 ГБ в сутки, 23 ГБ за восемь недель: в предел он не помещается.

В `SOURCE_FILTER` типы событий пишутся через запятую без пробелов.

sensor.community

Параметр	Значение
Поток	`https://data.sensor.community/static/v2/data.json` — замеры всех датчиков за последние 5 минут; `https://data.sensor.community/airrohr/v1/filter/country=<код>` — то же с фильтром по стране на стороне сервера
История	`https://archive.sensor.community/<ГГГГ-ММ-ДД>/` — файл CSV `<дата>_<тип>_sensor_<id>.csv` на датчик и сутки, с 2015 года, появляется на следующие сутки; колонки — `sensor_id`, `timestamp`, координаты и значения замера. Страны в архиве нет: загружаются файлы датчиков, которые поток отдаёт для стран задания (сутки DE — около 7000 файлов и часа последовательной загрузки). Строки приводятся к схеме события
Лицензия и условия	ODbL 1.0: при публичном использовании указывается источник, производные базы — под той же лицензией
Лимиты	не объявлены; данные обновляются раз в 5 минут — опрос не чаще
Событие	замер датчика; ключ — `sensor.id` и `timestamp`: `id` замера в архиве нет; время события — `timestamp` (UTC); страна — `location.country`

Таблица 3.7: sensor.community.

Известные дефекты: окна соседних опросов перекрываются — один замер приходит дважды; у 0,3 % замеров пара «датчик — время» повторяется с другими значениями; значения — строками; выбросы и датчики в помещении; неверные координаты у части датчиков.

Фильтр задания (`SOURCE_FILTER`)	Замеров в минуту	ГБ в сутки сжатых	ГБ за 8 недель
`DE`	1410	0,086	4,8
`NL`	540	0,033	1,9
`PL,CZ,AT`	370	0,023	1,3
`BG,RO,RS`	310	0,019	1,1
`RU,BY,KZ`	110	0,007	0,4

Таблица 3.8: sensor.community: фильтры и объёмы при опросе раз в 5 минут; сжатие JSONL ×7,6.

Все страны без фильтра — 3570 замеров в минуту, 0,22 ГБ в сутки, 12,2 ГБ за восемь недель.

adsb.lol

Параметр	Значение
Поток	`https://api.adsb.lol/v2/point/<широта>/<долгота>/<радиус>` — текущие положения воздушных судов в круге радиусом до 250 морских миль
История	суточный архив проекта: релиз `v<ГГГГ.ММ.ДД>-planes-readsb-prod-0` репозитория `adsblol/globe_history_<год>` на GitHub, две части tar, около 3,7 ГБ на сутки по всему миру, появляется на следующие сутки. Внутри — трек на борт `traces/<xx>/trace_full_<hex>.json`, сжатый gzip: точки со временем от начала суток, координатами, высотой, скоростью и курсом. Точки приводятся к схеме события, регион — фильтр по расстоянию до центра круга; архив читается потоком, на диск ложится только отфильтрованное
Лицензия и условия	поток — CC0 по странице лицензии проекта; суточный архив — ODbL 1.0 по странице исторических данных: при публичном использовании указывается источник, производные базы — под той же лицензией. Проект предупреждает, что доступ к API будет выдаваться по ключу
Лимиты	динамические: ответ 429 уже на втором запросе подряд; опрос не чаще раза в секунду, рабочая частота — раз в 10 с
Событие	положение борта; ключ — адрес `hex` и время положения; время события — `now` ответа минус `seen_pos`

Таблица 3.9: adsb.lol.

Известные дефекты: над Россией приёмников почти нет — регион задаёт задание; позывной `flight` пуст или с пробелами; положение без координат при `seen_pos` больше минуты.

Фильтр задания (`SOURCE_FILTER`)	Бортов в ответе	ГБ в сутки сжатых	ГБ за 8 недель
`50.03,8.57,250` — Франкфурт-на-Майне	340	0,22	12,3
`41.26,28.74,250` — Стамбул	100–130	0,07	3,9
`25.25,55.36,250` — Дубай	70	0,05	2,8

Таблица 3.10: adsb.lol: фильтры и объёмы при опросе раз в 10 с; сжатие JSON ×8.

Без фильтра источник не собирается: поток задаётся регионом.

GDELT

Параметр	Значение
Поток	`https://data.gdeltproject.org/gdeltv2/lastupdate.txt` раз в 15 минут называет три новых файла: `export` (события), `mentions` (упоминания), `gkg` (граф знаний)
История	файлы версии 2 сплошь с 18 февраля 2015 года, список — `masterfilelist.txt`: те же события и ключ
Лицензия и условия	свободное использование, в том числе коммерческое, при ссылке на GDELT Project
Лимиты	не объявлены; файлы загружаются по одному, не чаще публикации
Событие	строка `export`; ключ — `GLOBALEVENTID`; время события — `DATEADDED` (UTC); строка `mentions` — ключ `GLOBALEVENTID` и `MentionIdentifier`

Таблица 3.11: GDELT.

Известные дефекты: файлы без заголовков, поля разделены табуляцией; дата события `SQLDATE` бывает старше даты добавления на дни; файл иногда появляется позже или отсутствует.

Фильтр задания (`SOURCE_FILTER`)	Строк в минуту	ГБ в сутки сжатых	ГБ за 8 недель
`export,mentions`	220	0,019	1,1
`export`	60	0,007	0,4

Таблица 3.12: GDELT: фильтры и объёмы; сжатие JSONL после фильтра ×9.

Все три таблицы без фильтра — 0,6 ГБ в сутки, 34 ГБ за восемь недель, больше предела.

Паспорт источника

Паспорт — файл `SOURCE.md` в рабочем репозитории. Заводится в работе 1 и обновляется при смене схемы источника, новом дефекте, замене источника. Объёмы измеряются на собранных данных за интервал наблюдения и пересчитываются на сутки, объём до фильтра — коротким замером всего потока (минуты хватает). Если фильтр применяется на стороне источника и весь поток сборщику не виден, объём до фильтра берётся из таблиц этого раздела с пометкой.

```
# Паспорт источника: <название>

## Доступ
- адрес потока и способ получения (опрос, подписка, файлы по расписанию)
- адрес исторического среза и способ загрузки
- глубина доступной истории
- схема истории: те же события или отдельный набор; чем закрывается пропуск

## Лицензия и условия использования
- лицензия данных, условия, требования к указанию источника

## Лимиты и представление клиента
- объявленные лимиты, частота опроса и заголовок User-Agent сборщика

## Схема события
| Поле | Тип | Смысл | Пример |
|---|---|---|---|
- ключ события: <поле или сочетание полей>, почему оно уникально
- время события: <поле>, часовой пояс, точность

## Фильтр потока и объёмы
- фильтр задания: <значение SOURCE_FILTER>
- интервал наблюдения: <с — по>
| | До фильтра | После фильтра |
|---|---|---|
| событий в минуту | | |
| байт в сутки до сжатия | | |
| байт в сутки после сжатия | | |
- восемь недель после фильтра и сжатия: <ГБ> при пределе <COLLECTOR_RAW_LIMIT_GB> ГБ
- ограничение, которое наступит первым при росте потока: <память, диск, сеть
  или время ответа>

## Известные дефекты
- пропуски, дубликаты, поздние события, изменения схемы источника:
  что найдено, когда, как обрабатывается
```

Форма паспорта источника.

Контракт сборщика данных

Сборщик данных — программа студента на Python. Стенд даёт только оболочку службы: образ с интерпретатором, точку входа и каталог слоя сырых данных. Реализации сборщика в стенде нет. Программа кладётся в `collector/src/` рабочего репозитория, точка входа — `main.py`, зависимости — строками в `collector/requirements.txt`.

Контракт ниже одинаков для всех источников. Предметная часть — как получить поток и историю, что считать ключом и временем события — берётся из раздела «Источники данных» и записывается в паспорт источника.

Режимы

Режим приходит первым аргументом программы и в переменной `COLLECTOR_MODE`.

Режим	Что делает	Как запускается	С работы
`record` — запись	получает поток источника, применяет фильтр задания и пишет события в слой сырых данных	служба стенда: `COLLECTOR_MODE=record` в `.env`	ЛР1
`backfill` — загрузка истории	получает исторический срез за период из переменных и пишет его в отдельный префикс слоя	разово: `docker compose run --rm collector backfill`	ЛР1
`produce` — отправка	делает всё, что `record`, и публикует каждое событие в тему брокера по схеме из реестра	служба стенда: `COLLECTOR_MODE=produce` в `.env`	ЛР6
`replay` — повтор	читает файлы слоя сырых данных за период и публикует их в брокер как поток с заданной скоростью	разово: `docker compose run --rm collector replay`	ЛР6

Таблица 4.1: Режимы сборщика.

В режиме `produce` недоступность брокера не останавливает запись: события ложатся в файлы, ошибки отправки видны в журнале, а пропущенное в брокере восполняется режимом `replay`.

С работы 3 режимы `record` и `produce` копируют каждый закрытый файл слоя в озеро — в префикс, заданный переменной `COLLECTOR_LAKE_PREFIX`, с тем же разбиением по времени. При пустой переменной копирование выключено. Файлы, ещё не скопированные к моменту запуска, докопируются при старте; повторная копия уже лежащего объекта не создаётся.

Переменные окружения

Сборщик настраивается только переменными окружения: адресов, учётных данных и параметров источника в коде нет. Переменные читаются из `.env` стенда. Все переменные, которые заводит сам студент, начинаются с `COLLECTOR_` и перечислены в `collector/src/README.md` с назначением и значением по умолчанию.

Переменная	Кто задаёт	Смысл
`SOURCE`, `SOURCE_FILTER`	`.env`, по заданию	источник и фильтр потока; пустой фильтр — весь поток
`COLLECTOR_RAW_LIMIT_GB`	`.env`, по заданию	предел объёма слоя сырых данных, ГБ
`COLLECTOR_MODE`	оболочка	режим, он же первый аргумент
`COLLECTOR_RAW_DIR`	оболочка	корень слоя сырых данных, `/data/raw`
`COLLECTOR_OUT_DIR`	оболочка	каталог вывода режима: `/data/raw/stream` или `/data/raw/history`
`COLLECTOR_METRICS_PORT`	оболочка	порт счётчиков, 9101
`S3_*`, `BROKER_BOOTSTRAP`, `REGISTRY_URL`, `SERDE`	`.env` стенда	адреса и учётные данные сервисов, способ описания схемы
`COLLECTOR_USER_AGENT`	студент	заголовок `User-Agent` с контактом
`COLLECTOR_POLL_SECONDS`	студент	период опроса источника
`COLLECTOR_HISTORY_FROM`, `COLLECTOR_HISTORY_TO`	студент	период загрузки истории, даты UTC
`COLLECTOR_REPLAY_FROM`, `COLLECTOR_REPLAY_TO`, `COLLECTOR_REPLAY_SPEED`	студент	период и скорость повтора
`COLLECTOR_TOPIC`	студент	тема брокера для отправки и повтора
`COLLECTOR_LAKE_PREFIX`	студент	префикс слоя сырых данных в озере

Таблица 4.2: Переменные окружения сборщика.

Слой сырых данных

- **Раскладка.** Поток пишется в `/data/raw/stream`, исторический срез — в `/data/raw/history`; префикс задаёт оболочка переменной `COLLECTOR_OUT_DIR`. Внутри префикса файлы разложены по времени: поток — по часу получения (`dt=ГГГГ-ММ-ДД/hour=ЧЧ/`), история — по дате события (`dt=ГГГГ-ММ-ДД/`). Поток раскладывается по времени получения, а не события: поздние события тогда не дописываются в закрытые часы.
- **Формат.** JSON Lines, одно событие в строке, файл сжат gzip (`*.jsonl.gz`). Файл пишется под временным именем и переименовывается при закрытии — по часу или по размеру. Незакрытых файлов под рабочим именем нет.

- **Поля события.** Каждое событие несёт `event_key` — идемпотентный ключ, `event_time` — время события в UTC (ISO 8601 с миллисекундами), `received_at` — время получения и `source` — значение оси. Остальные поля — поля события источника по схеме из паспорта. У истории со своей схемой (свечи MOEX ISS) те же четыре поля несёт каждая запись истории: ключ и время берутся из неё.
- **Фильтр.** Событий вне фильтра задания в слое нет. Фильтр, который источник умеет применять сам, передаётся источнику; остальное отбрасывает сборщик.
- **Перезапуск.** Позиция в источнике (смещение, время последнего опроса, последний загруженный файл) сохраняется в `/data/raw/state/` после закрытия файла. После остановки и запуска сборщик продолжает с сохранённой позиции: ничего не теряется, а события перекрытия отбрасываются по ключу.
- **Предел и ротация.** Сумма файлов `stream` и `history` не превышает `COLLECTOR_RAW_LIMIT_GB`. При достижении предела удаляются самые старые закрытые файлы; каталог `state` в предел не входит. Предел рассчитан так, чтобы вмещать не меньше восьми недель потока после фильтра и сжатия; расчёт приводится в паспорте источника.
- **Частота.** Обращения к источнику не чаще объявленного предела (раздел «Источники данных»); ответы 429 и 5xx — повтор с растущей паузой.
- **Память.** Служба работает с пределом 512 МБ без подкачки: ответ источника и архив истории читаются потоком, а не целиком. Превышение предела завершает процесс с кодом 137, и служба перезапускается.

Журнал

Журнал пишется в стандартный вывод строками JSON — по строке на событие программы — и читается командой `docker compose logs collector`. Обязательные поля: `ts` (время записи, UTC), `level`, `msg`; остальные — по смыслу события. Строка запуска называет режим.

```
{"ts":"2026-01-12T10:00:01Z","level":"info","msg":"start","mode":"record"}
{"ts":"2026-01-12T10:00:11Z","level":"info","msg":"poll","got":412,"kept":57}
{"ts":"2026-01-12T10:00:21Z","level":"warn","msg":"http","status":429,"pause":8}
{"ts":"2026-01-12T10:00:29Z","level":"info","msg":"retry","attempt":2}
{"ts":"2026-01-12T11:00:00Z","level":"info","msg":"closed","lines":9870}
{"ts":"2026-01-12T11:00:00Z","level":"info","msg":"rotated","files":1}
```

Строки журнала сборщика.

Журнал показывает поведение при ошибке источника: код ответа, паузу перед повтором и восстановление. Строка опроса называет число событий до фильтра и после него: из журнала паспорт берёт долю потока, которую оставляет фильтр.

Счётчики

Сборщик отдаёт счётчики по HTTP на порту `COLLECTOR_METRICS_PORT` (9101), путь `/metrics`, в текстовом формате Prometheus: число собранных событий, число ошибок обращения к источнику и время последнего успешного обращения.

```
collector_events_total{mode="record"} 1834023
collector_errors_total{mode="record",kind="http"} 17
collector_last_success_timestamp_seconds 1768212011
```

Счётчики сборщика.

Счётчики — дополнительное задание работы 1.

Запуск

Запуск программы службой — шаг по готовому файлу: оболочка уже описана в файлах стенда.

```
docker compose build collector              # после правки requirements.txt
docker compose up -d collector              # служба в режиме COLLECTOR_MODE
docker compose restart collector            # после правки программы
docker compose logs -f collector
docker compose run --rm collector backfill  # разовые режимы
docker compose run --rm -e COLLECTOR_REPLAY_SPEED=10 collector replay
```

Программа смонтирована в контейнер только для чтения: правка кода вступает в силу после перезапуска службы. Служба перезапускается сама после сбоя и после перезагрузки стенда.

Сервисы стенда

Справочник по сервисам стенда: что стоит за каждой ролью, как к ней подключиться и чем реализации одной роли отличаются друг от друга. Имя сервиса — роль, а не продукт: какой продукт стоит за ролью у варианта, говорит таблица параметров задания.

Адреса и учётные данные

Изнутри стенда — из контейнеров и заданий движков — сервисы доступны по имени роли, из программ, запущенных на стенде вне контейнеров, — по `localhost` и порту из раздела «Стенд курса». Веб-интерфейсы открываются в браузере рабочей машины по адресам того же раздела. Клиенты брокера — программы на стенде: брокер сообщает клиенту адрес `localhost`, и с рабочей машины подключение работает только на WSL2.

Роль	Изнутри стенда	Учётные данные
объектное хранилище	`http://s3:9000`, bucket `lake`, регион `stand`	ключ `studentaccess`, секрет `studentsecretkey`
каталог таблиц	`http://catalog:8181/catalog`, склад `lake`	без входа
аналитическое хранилище	`warehouse:8123` (HTTP), `warehouse:9000` (родной)	`student` / `student`, база `mart`
брокер	`broker:9092`	без входа
реестр схем	`http://registry:8081`	без входа

Таблица 5.1: Адреса и учётные данные сервисов.

Учётные данные стенда живут в `.env`, а он в рабочий репозиторий не входит. Программы на Python читают их из переменных окружения, файлы подключений `conf/` — ссылкой `${ENV:ИМЯ}` (перечень переменных — `.env.example`). Сборщику, `spark`, `duckdb` и потоковому заданию Spark виден весь `.env`, движку Trino и оркестратору — только `S3_*`, `CATALOG_*` и `WAREHOUSE_*`. В SQL-клиентах без подстановки переменных — StarRocks, ClickHouse, Flink — пишутся значения из таблицы.

Объектное хранилище

Роль `s3`. Bucket `lake` создаётся сам при первом запуске профиля работы 3.

	SeaweedFS	Garage
API S3	порт 9000	порт 9000
Порт 9001	веб-интерфейс; bucket — каталог `/buckets/lake`	административный API; веб-интерфейса нет
Регион в подписи запроса	не проверяется	проверяется: без региона `stand` — ответ 403
Клиенты на Java AWS SDK: Spark, Flink, StarRocks	работают без настройки	требуют переменных контрольных сумм; в файлах стенда они заданы

Таблица 5.2: Реализации объектного хранилища.

Переменные контрольных сумм — `AWS_REQUEST_CHECKSUM_CALCULATION` и `AWS_RESPONSE_CHECKSUM_VALIDATION` со значением `when_required`. Проверка bucket'а из командной строки:

```
docker compose exec s3 /garage bucket info lake              # Garage
echo s3.bucket.list | docker compose exec -T s3 \
    weed shell -master=s3:9333 -filer=s3:9001                 # SeaweedFS
```

Различия реализаций интерфейса S3 — это различия обработки подписи и контрольных сумм: клиент, правильно работающий с одной реализацией, может получить отказ от другой. Поэтому регион указывается всегда, даже там, где он не проверяется.

Программа на Python обращается к хранилищу через `boto3`:

```
import os
import boto3
from botocore.config import Config

s3 = boto3.client(
    "s3",
    endpoint_url=os.environ["S3_ENDPOINT"],        # вне контейнеров — localhost:9000
    region_name=os.environ["S3_REGION"],
    aws_access_key_id=os.environ["S3_ACCESS_KEY"],
    aws_secret_access_key=os.environ["S3_SECRET_KEY"],
    config=Config(s3={"addressing_style": "path"}),
)
s3.upload_file("part.jsonl.gz", "lake", "raw/.../part.jsonl.gz")
```

Клиент объектного хранилища.

`upload_file` сам переходит на многочастную загрузку для больших файлов. Листинг по префиксу — `list_objects_v2` с продолжением по `ContinuationToken`: один ответ содержит не больше тысячи ключей.

Встраиваемый движок

DuckDB — встраиваемый движок без сервера: запускается командой на время работы, а не службой. Он одинаков у всех вариантов в работах 2 и 3 и служит движком запросов у варианта DuckDB с работы 4.

```
docker compose run --rm duckdb                    # интерактивно
docker compose run --rm -T duckdb < jobs/q.sql    # файл запросов
docker compose run --rm duckdb -init init.sql     # с файлом подключений
```

Рабочий каталог — `/data/duckdb` (на стенде `data/duckdb`), слой сырых данных смонтирован только для чтения в `/data/raw`. Расширения `iceberg`, `httpfs` и `avro` установлены в образ, сеть для них не нужна. Время запросов показывает `.timer on`, план с числами — `EXPLAIN ANALYZE`.

Подключение к объектному хранилищу и к каталогу таблиц — секретами. Секреты держатся в файле `data/duckdb/init.sql`: он вне рабочего репозитория, и учётные данные в репозиторий не попадают.

```
LOAD httpfs;
LOAD iceberg;
CREATE SECRET s3sec (TYPE s3, KEY_ID 'studentaccess', SECRET 'studentsecretkey',
    ENDPOINT 's3:9000', URL_STYLE 'path', USE_SSL false, REGION 'stand');
CREATE SECRET ic (TYPE iceberg, TOKEN '', ENDPOINT 'http://catalog:8181/catalog');
ATTACH 'lake' AS lake (TYPE iceberg, SECRET ic, ACCESS_DELEGATION_MODE 'none');
```

Подключения DuckDB к хранилищу и каталогу.

Без `ACCESS_DELEGATION_MODE 'none'` DuckDB ждёт учётных данных от каталога, не получает их и обращается к хранилищу без подписи — ответ 403.

Что умеет DuckDB в курсе:

- читать сжатые JSON Lines и Parquet с диска и из хранилища, в том числе с разбором партиций из путей;
- писать Parquet с партициями на диск и в хранилище; с `OVERWRITE_OR_IGNORE` повторный запуск заменяет файлы партиций, а не добавляет копии. Заменяется файл каждой партиции, в которую пишет запуск: запуск за сутки D, записавший опоздавшие события суток D−1, стирает партицию D−1. JSON с партициями DuckDB не пишет: слой сырых данных переносится в озеро программой на `boto3`;
- создавать таблицы Iceberg, в том числе с партициями, и писать в них после `ATTACH`: `INSERT`, `UPDATE`, `DELETE`;
- показывать метаданные таблицы: снимки и файлы данных с числом строк.

Файлы слоя сырых данных разложены по каталогам `dt=…/hour=…`, и `read_json_auto` добавляет к полям события колонки `dt` и `hour` из пути (`hive_partitioning = false` это отключает). Ключ партиции очищенных данных поэтому называется иначе — `event_date`: запись с `PARTITION_BY (dt)` при колонке `dt` из пути создаёт каталоги `dt_1=…`.

Сутки, которые обрабатывает файл запросов, задаются его первой строкой. Шаг конвейера задаёт ту же переменную до запуска файла, и значение из файла тогда не действует. Шаблон `raw/*/dt=…/**` берёт сутки и из потока, и из исторического среза: история закрывает пропуск в записи теми же событиями, а повторы отбрасываются очисткой по ключу. У MOEX ISS история — свечи, отдельный набор, поэтому шаблон суток — `raw/stream/dt=…`. Шаблон, под который не подошёл ни один файл, завершает запрос ошибкой: без сырья суток D+1 запрос не выполнится.

```
SET VARIABLE day = coalesce(getvariable('day'), DATE '<ГГГГ-ММ-ДД>');
SELECT count(*) FROM read_json_auto([
    's3://lake/raw/*/dt=' || getvariable('day') || '/**/*.jsonl.gz',
    's3://lake/raw/*/dt=' || (getvariable('day') + 1) || '/**/*.jsonl.gz'])
WHERE CAST(event_time AS DATE) = getvariable('day');
```

Сутки как параметр файла запросов: события суток D из сырья D и D+1, потока и истории.

```
SELECT count(*) FROM read_json_auto('s3://lake/raw/stream/**/*.jsonl.gz');
SELECT count(*) FROM read_parquet('s3://lake/clean/events/*/*.parquet',
    hive_partitioning = true);
COPY (SELECT *, CAST(event_time AS DATE) AS event_date FROM t)
    TO 's3://lake/clean/events'
    (FORMAT parquet, PARTITION_BY (event_date), OVERWRITE_OR_IGNORE true);
CREATE TABLE lake.clean.t (event_key VARCHAR, event_time TIMESTAMP)
    PARTITIONED BY (day(event_time));
SELECT * FROM iceberg_snapshots('lake.clean.t');
SELECT * FROM iceberg_metadata('lake.clean.t');
```

DuckDB: файлы озера и таблицы Iceberg.

Каталог таблиц

Роль `catalog` — Lakekeeper, каталог Iceberg с интерфейсом REST, одинаковый у всех вариантов. Склад `lake` над bucket'ом `lake` создаётся сам при первом запуске профиля работы 4. Склад не выдаёт движкам учётных данных хранилища: каждый движок ходит в хранилище своим ключом.

Пространство имён таблиц курса — `clean`. Таблицы видны любому движку, который подключён к каталогу: у Spark, Trino и DuckDB каталог называется `lake`, у ClickHouse — база, созданная над каталогом (подраздел «Аналитическое хранилище»).

Файлы таблицы лежат в bucket'е под её идентификатором, а не под именем: данные — в `lake/<идентификатор таблицы>/data/`, метаданные — рядом, в `metadata/`. Путь к файлам поэтому берётся из метаданных таблицы, а не из листинга bucket'а. Удаление файлов мимо таблицы ломает её: таблица ссылается на файл, которого больше нет.

Движок запросов

Роль `engine` — ось `engine`. Движок запрашивает таблицы Iceberg. Пишут в таблицы Spark и DuckDB, обслуживает их Spark (подраздел «Пакетный движок и обслуживание таблиц»).

Trino. Веб-интерфейс — `http://localhost:8080`, вход под любым именем без пароля. Каталог Trino — файл `conf/trino/<каталог>.properties` рабочего репозитория: имя файла — имя каталога, строка — свойство `ключ=значение`. Каталог `lake` описывает файл `conf/trino/lake.properties`:

Свойство	Значение
`connector.name`, `iceberg.catalog.type`	коннектор `iceberg`, тип каталога `rest`
`iceberg.rest-catalog.uri`, `iceberg.rest-catalog.warehouse`	адрес каталога таблиц и склад
`fs.native-s3.enabled`	`true`: встроенный клиент S3
`s3.endpoint`, `s3.region`	адрес объектного хранилища и регион подписи

`s3.path-style-access`	`true`: bucket — часть пути
`s3.aws-access-key`, `s3.aws-secret-key`	ключ и секрет объектного хранилища

Таблица 5.3: Свойства каталога lake в Trino.

Trino читает каталоги при запуске, поэтому после правки файла сервис перезапускается. С ошибкой в файле он перезапускается по кругу, причину называет строка `ERROR` журнала.

```
docker compose restart engine
docker compose logs engine | grep -A3 ERROR
docker compose exec engine trino --catalog lake --schema clean
```

Сколько прочитано, показывает `EXPLAIN ANALYZE`: у оператора `TableScan` — число разбиений (по файлу данных на разбиение), строки и байты. Метаданные — служебные таблицы `"t$snapshots"`, `"t$files"`, `"t$partitions"`; состояние на прошлый момент — `FOR VERSION AS OF` и `FOR TIMESTAMP AS OF`.

StarRocks. Один управляющий и один вычислительный узел в одном контейнере с пределами памяти стенда. Веб-интерфейс — `http://localhost:8080` (пользователь `root` без пароля), SQL — протокол MySQL на порту 9030. Каталог Iceberg создаётся один раз:

```
docker compose exec engine mysql -h127.0.0.1 -P9030 -uroot
```

```
CREATE EXTERNAL CATALOG lake PROPERTIES (
  'type' = 'iceberg',
  'iceberg.catalog.type' = 'rest',
  'iceberg.catalog.uri' = 'http://catalog:8181/catalog',
  'iceberg.catalog.warehouse' = 'lake',
  'aws.s3.endpoint' = 'http://s3:9000',
  'aws.s3.enable_path_style_access' = 'true',
  'aws.s3.access_key' = 'studentaccess',
  'aws.s3.secret_key' = 'studentsecretkey',
  'aws.s3.region' = 'stand'
);
SET CATALOG lake;
USE clean;
```

Каталог Iceberg в StarRocks.

Сколько прочитано, показывает профиль запроса. Сбор профиля включается перед запросом. Профиль запроса дольше секунды готов не сразу: идентификатор запроса запоминается, профиль читается через несколько секунд.

```
SET enable_profile = true;
SELECT count(DISTINCT event_key) FROM t
WHERE event_time >= '<ГГГГ-ММ-ДД> 00:00:00';
SELECT last_query_id();
SELECT get_query_profile('<идентификатор запроса>');
```

В строке `ScanReport` профиля — файлы данных после отсечения (`resultDataFiles`), пропущенные манифесты (`skippedDataManifests`) и объём этих файлов (`totalFileSizeInBytes`); у оператора сканирования — диапазоны сканирования (`ScanRanges`)

и строки (`RawRowsRead`). Повторный запрос читает из кэша данных, поэтому сравнение ведётся по файлам, их объёму и строкам, а не по байтам ввода-вывода. Запрос `count(*)` без фильтра строк не читает: число строк берётся из метаданных. StarRocks не пишет в таблицу с партицией по колонке времени: запись идёт через Spark или DuckDB.

DuckDB. Встраиваемый движок без порта (подраздел «Встраиваемый движок»). Сколько прочитано, показывает `EXPLAIN ANALYZE`: у сканирования — число и имена прочитанных файлов (`Total Files Read`) и строки, в блоке `HTTPFS HTTP Stats` — байты, принятые из хранилища. Все файлы таблицы с числом строк перечисляет `iceberg_metadata()`.

Пакетный движок и обслуживание таблиц

Роль `spark` — Spark с Iceberg, одинаковый у всех вариантов. Он же — единый инструмент обслуживания таблиц: компакция, устаревание снимков, откат и эволюция схемы идут через него у всех вариантов. Сервис запускается на время работы. Подключение к каталогу `lake` — файл `conf/spark/spark-defaults.conf` рабочего репозитория, строка — свойство и значение через пробелы:

Свойство	Значение
`spark.sql.catalog.lake`	класс каталога `org.apache.iceberg.spark.SparkCatalog`
`spark.sql.catalog.lake.type`	тип каталога `rest`
`spark.sql.catalog.lake.uri`, `spark.sql.catalog.lake.warehouse`	адрес каталога таблиц и склад
`spark.sql.catalog.lake.io-impl`	клиент хранилища `org.apache.iceberg.aws.s3.S3FileIO`
`spark.sql.catalog.lake.s3.endpoint`, `spark.sql.catalog.lake.client.region`	адрес объектного хранилища и регион подписи
`spark.sql.catalog.lake.s3.path-style-access`	`true`: bucket — часть пути
`spark.sql.catalog.lake.s3.access-key-id`, `spark.sql.catalog.lake.s3.secret-access-key`	ключ и секрет объектного хранилища
`spark.sql.defaultCatalog`	`lake`: каталог для имён таблиц без каталога

Таблица 5.4: Свойства подключения Spark к каталогу lake.

Файл общий для `spark` и потокового задания Spark (`stream`). Он читается при каждом запуске сервиса, ссылки `${ENV:ИМЯ}` подставляются тогда же; переменная, которой нет в `.env`, останавливает запуск с номером строки. После строк файла стенд дописывает свои — расширения Iceberg, память драйвера, число партиций при перемешивании, — и при совпадении свойства действует строка стенда. Без файла Spark запускается, но каталога `lake` не видит.

```
1  docker compose run --rm --service-ports spark /opt/spark/bin/spark-sql
2  docker compose run --rm --service-ports spark \
3      /opt/spark/bin/spark-submit /opt/jobs/aggregate.py
```

Задания лежат в каталоге `jobs/` рабочего репозитория (в контейнере — `/opt/jobs`). Интерфейс задания — `http://localhost:4040`, пока задание идёт; план запроса — `EXPLAIN FORMATTED` или `df.explain("formatted")`. Второй одновременный запуск идёт без `--service-ports`:

порт 4040 занят. Предупреждение `WARN S3FileIO: Unclosed S3FileIO instance` в конце вывода — не ошибка.

```
SELECT snapshot_id, committed_at, operation FROM lake.clean.t.snapshots;
SELECT count(*), sum(file_size_in_bytes) FROM lake.clean.t.files;
SELECT count(*) FROM lake.clean.t VERSION AS OF 1234567890123;
CALL lake.system.rollback_to_snapshot('clean.t', 1234567890123);
CALL lake.system.rewrite_data_files(table => 'clean.t');
CALL lake.system.expire_snapshots(table => 'clean.t',
    older_than => TIMESTAMP '<ГГГГ-ММ-ДД> 00:00:00', retain_last => 1);
ALTER TABLE lake.clean.t ADD COLUMN lang STRING;
```

Обслуживание таблицы Iceberg в Spark.

Компакция `rewrite_data_files` с параметрами по умолчанию переписывает партицию, в которой не меньше пяти файлов данных (параметр `min-input-files`); партиция из одного файла остаётся как есть.

Повторный счёт суток заменяет только их партицию при одном из способов записи:

- метод `overwritePartitions()` у `df.writeTo(<таблица>)`;
- `INSERT OVERWRITE` после `SET spark.sql.sources.partitionOverwriteMode=dynamic`.

В режиме по умолчанию `INSERT OVERWRITE` без условия на партицию заменяет таблицу целиком: после пересчёта одних суток остальные исчезают.

Аналитическое хранилище

Роль `warehouse` — ClickHouse, одинаковый у всех вариантов. Интерфейс запросов в браузере — `http://localhost:8123/play`.

```
docker compose exec warehouse clickhouse-client -u student --password student
```

Файл запросов выполняется через стандартный ввод. Сутки передаются параметром, в файле они стоят как `{day:Date}`:

```
docker compose exec -T warehouse clickhouse-client -u student --password student \
    --param_day=<ГГГГ-ММ-ДД> < jobs/stg_load.sql
```

В шаге конвейера тот же файл выполняется пакетом `clickhouse-connect` по одному запросу, сутки — аргументом `parameters={"day": day}`.

Файлы озера ClickHouse читает функцией `s3`, таблицы Iceberg — через базу над каталогом (только чтение):

```
SELECT count() FROM s3('http://s3:9000/lake/clean/<набор>/*/*.parquet',
    'studentaccess', 'studentsecretkey', 'Parquet');

CREATE DATABASE lake_ice ENGINE = DataLakeCatalog('http://catalog:8181/catalog',
    'studentaccess', 'studentsecretkey')
SETTINGS catalog_type = 'rest', storage_endpoint = 'http://s3:9000/lake',
    warehouse = 'lake';
SELECT count() FROM lake_ice.`clean.t`;
```

ClickHouse: файлы озера и таблицы каталога.

Запрос к базе над каталогом читает текущий снимок таблицы: новые коммиты видны без пересоздания базы. Регион подписи для хранилища задан в конфигурации сервера стенда. Программы на Python подключаются к ClickHouse пакетом `clickhouse-connect`.

Брокер и реестр схем

Роль `broker` — ось `broker`, роль `registry` — реестр схем. Изнутри стенда брокер отвечает по `broker:9092`, программам на стенде вне контейнеров — по `localhost:9092`. Реестр у обоих значений оси отвечает по `registry:8081` и `localhost:8081` с одним и тем же API.

Реестр схем у Apache Kafka — Karapace, отдельный сервис `registry`; у Redpanda реестр встроен в брокер, а имя `registry` ведёт на него же. Темы и группы потребителей — утилитами брокера:

```
# Apache Kafka: утилиты лежат в /opt/kafka/bin контейнера
docker compose exec broker /opt/kafka/bin/kafka-topics.sh \
    --bootstrap-server localhost:9092 --create --topic <тема> \
    --partitions <N> --replication-factor 1
docker compose exec broker /opt/kafka/bin/kafka-topics.sh \
    --bootstrap-server localhost:9092 --list
docker compose exec broker /opt/kafka/bin/kafka-consumer-groups.sh \
    --bootstrap-server localhost:9092 --describe --group <группа>
# Redpanda
docker compose exec broker rpk topic create <тема> -p <N>
docker compose exec broker rpk topic list
docker compose exec broker rpk group describe <группа>
```

Число сообщений в партиции — разность конечного и начального смещений, объём партиции — размер её журнала на диске брокера:

```
# Apache Kafka: конечное смещение по партициям; --time earliest — начальное
docker compose exec broker /opt/kafka/bin/kafka-get-offsets.sh \
    --bootstrap-server localhost:9092 --topic <тема>
docker compose exec broker /opt/kafka/bin/kafka-log-dirs.sh \
    --bootstrap-server localhost:9092 --describe --topic-list <тема>
# Redpanda: LOG-START-OFFSET и HIGH-WATERMARK по партициям; объём партиций
docker compose exec broker rpk topic describe <тема> -p
docker compose exec broker rpk cluster logdirs describe --topics <тема>
```

Реестр работает по API реестра схем Confluent; по умолчанию режим совместимости — `BACKWARD`. Проверка совместимости отвечает 200 с полем `is_compatible`; ответ 409 даёт регистрация несовместимой схемы. Схема Protobuf передаётся с полем `"schemaType": "PROTOBUF"`: без него реестр разбирает её как Avro и отвечает 422.

```
GET  /subjects                                        список subject'ов
POST /subjects/<тема>-value/versions                  регистрация схемы
POST /compatibility/subjects/<тема>-value/versions/latest   проверка
GET  /config/<тема>-value, PUT /config/<тема>-value   режим совместимости
тело регистрации и проверки: {"schema": "<схема строкой>"}
                             у Protobuf ещё "schemaType": "PROTOBUF"
```

API реестра схем.

Сообщение в формате реестра — магический байт, четыре байта идентификатора схемы и тело (у Protobuf между ними ещё индекс сообщения). Клиент на Python — пакет `confluent-kafka` с

сериализаторами реестра для Avro и Protobuf.

Оркестратор

Роль `orchestrator` — ось `orchestrator`. Веб-интерфейс — `http://localhost:8082`. Конвейер лежит в каталоге `dags/` рабочего репозитория. В образе оркестратора есть Spark с Iceberg (локальный режим), DuckDB с расширениями `iceberg`, `httpfs` и `avro` и клиент ClickHouse: шаги конвейера выполняются внутри оркестратора. Запросы и задания шагов — те же файлы `jobs/`, что запускаются вручную: оркестратор видит каталог как `/opt/jobs`, и модули из него импортируются в шагах напрямую. Метаданные запусков хранятся в PostgreSQL каталога таблиц. Задачи Airflow и запуски Dagster выполняются по одному: у каждого шага Spark своя JVM.

Файл `data/duckdb/init.sql` оркестратору не виден: шаг DuckDB создаёт секреты из переменных окружения, задаёт сутки запуска и выполняет файл запросов.

```
import os
import duckdb

e = os.environ
con = duckdb.connect()
con.execute("LOAD httpfs; LOAD iceberg")
con.execute(f"""CREATE SECRET s3sec (TYPE s3, KEY_ID '{e["S3_ACCESS_KEY"]}',
    SECRET '{e["S3_SECRET_KEY"]}', ENDPOINT 's3:9000', URL_STYLE 'path',
    USE_SSL false, REGION '{e["S3_REGION"]}')""")
# секрет каталога и ATTACH — как в init.sql, если шаг пишет в таблицу
con.execute("SET VARIABLE day = ?", [day])      # сутки запуска
con.execute(open("/opt/jobs/clean.sql").read())
```

Шаг DuckDB внутри оркестратора.

Apache Airflow 3. Вход без пароля. Файл с DAG'ом кладётся в `dags/`, оркестратор перечитывает каталог раз в 30 с. Задачи описываются декораторами `@dag` и `@task` из `airflow.sdk`. Новый DAG появляется выключенным: ни расписание, ни повторный счёт не запускаются, пока DAG не включён переключателем в интерфейсе или командой `airflow dags unpause <dag>`. Включённый DAG со `start_date` в прошлом сразу получает последний пропущенный запуск по расписанию; без `start_date` первый запуск — в ближайший срок расписания. Повторный счёт за прошедший период:

```
docker compose exec orchestrator airflow backfill create --dag-id <dag> \
    --from-date <ГГГГ-ММ-ДД> --to-date <ГГГГ-ММ-ДД> --max-active-runs 1
```

Dagster. Вход не нужен. Определения — активы, задания и расписания — читаются из `dags/pipeline.py`. Расписание по умолчанию выключено: оно включается в интерфейсе или объявляется с `default_status=DefaultScheduleStatus.RUNNING`. Повторный счёт — активы с суточными партициями и запуск по диапазону партиций из интерфейса.

Сессия Spark в шаге конвейера `conf/spark/` не читает, свойства задаются в коде:

```
import os
from pyspark.sql import SparkSession

e = os.environ
spark = (
    SparkSession.builder.master("local[2]").appName("aggregate")
    .config("spark.driver.memory", "1g")
    .config("spark.sql.extensions",
            "org.apache.iceberg.spark.extensions.IcebergSparkSessionExtensions")
    .config("spark.sql.catalog.lake", "org.apache.iceberg.spark.SparkCatalog")
    .config("spark.sql.catalog.lake.type", "rest")
    .config("spark.sql.catalog.lake.uri", e["CATALOG_URI"])
    .config("spark.sql.catalog.lake.warehouse", e["CATALOG_WAREHOUSE"])
    .config("spark.sql.catalog.lake.io-impl", "org.apache.iceberg.aws.s3.S3FileIO")
    .config("spark.sql.catalog.lake.s3.endpoint", e["S3_ENDPOINT"])
    .config("spark.sql.catalog.lake.s3.path-style-access", "true")
    .config("spark.sql.catalog.lake.s3.access-key-id", e["S3_ACCESS_KEY"])
    .config("spark.sql.catalog.lake.s3.secret-access-key", e["S3_SECRET_KEY"])
    .config("spark.sql.catalog.lake.client.region", e["S3_REGION"])
    .getOrCreate()
)
```

Сессия Spark внутри оркестратора.

Потоковый движок

Роль `stream` — ось `stream`. Задания лежат в каталоге `jobs/` рабочего репозитория (в контейнере — `/opt/jobs`), состояние — на диске стенда, поэтому перезапуск стенда не обнуляет накопленное.

Приёмник Iceberg фиксирует данные на каждом checkpoint'е: каждая фиксация — снимок таблицы и новые файлы. Поэтому фиксация идёт не чаще раза в минуту, а таблица потока раз в сутки обслуживается командами подраздела «Пакетный движок и обслуживание таблиц»: компакция и устаревание снимков.

Apache Flink 2.1. Служба из двух контейнеров: `stream` (управление, веб-интерфейс `http://localhost:8083`) и `stream-worker` (исполнение). Задание на SQL запускается клиентом внутри контейнера; checkpoint'ы пишутся раз в минуту в `data/stream/flink/checkpoints` и сохраняются при отмене задания.

```
docker compose exec stream ./bin/sql-client.sh -f /opt/jobs/stream.sql
```

```
CREATE CATALOG lake WITH (
  'type' = 'iceberg', 'catalog-type' = 'rest',
  'uri' = 'http://catalog:8181/catalog', 'warehouse' = 'lake',
  'io-impl' = 'org.apache.iceberg.aws.s3.S3FileIO',
  's3.endpoint' = 'http://s3:9000', 's3.path-style-access' = 'true',
  's3.access-key-id' = 'studentaccess', 's3.secret-access-key' = 'studentsecretkey',
  'client.region' = 'stand'
);
CREATE TABLE events_src (
  event_key STRING,
  event_time TIMESTAMP(3),
  WATERMARK FOR event_time AS event_time - INTERVAL '30' SECOND
) WITH (
  'connector' = 'kafka', 'topic' = '<тема>',
  'properties.bootstrap.servers' = 'broker:9092',
  'properties.group.id' = 'stream', 'scan.startup.mode' = 'earliest-offset',
  'format' = 'avro-confluent', 'avro-confluent.url' = 'http://registry:8081',
  'scan.watermark.idle-timeout' = '1 min'
);
```

Каталог Iceberg и таблица-источник в задании Flink SQL.

Время события схемы ЛР2 — `long` с логическим типом `timestamp-millis` — формат `avro-confluent` читает как `TIMESTAMP(3)`. Опция `scan.watermark.idle-timeout` исключает из watermark партицию темы, в которой минуту нет событий: без неё пустая партиция держит watermark задания, и окна не закрываются. Задание после остановки стенда продолжается с последнего checkpoint'а — перед `INSERT` задаётся путь к нему:

```
SET 'execution.state-recovery.path' =
    'file:///opt/flink/state/checkpoints/<идентификатор задания>/chk-<N>';
```

Spark Structured Streaming. Отдельного сервера нет: задание запускается в сервисе `stream` тем же образом, что и пакетный Spark, и работает, пока его не остановят. Интерфейс задания — `http://localhost:8083`. Checkpoint задаётся опцией `checkpointLocation` в каталоге `/home/spark/state` (на стенде `data/stream/spark`), фиксация раз в минуту — `trigger(processingTime="1 minute")`. Команда `docker compose down` задание не останавливает: перед ней задание удаляется командой `docker rm -f stream-job`.

```
docker compose run -d --name stream-job --service-ports stream \
    /opt/spark/bin/spark-submit /opt/jobs/stream.py
docker logs -f stream-job
```

Сообщения в формате реестра читаются функциями `from_avro` и `from_protobuf` после среза заголовка: пять байт у Avro, шесть у Protobuf с одним сообщением в файле схемы. Схема Avro берётся из реестра, для Protobuf — файл дескриптора, собранный компилятором из пакета `grpcio-tools` в каталог `jobs/`, который виден заданию:

```
python -m grpc_tools.protoc -I schemas --include_imports \
    --descriptor_set_out=jobs/event.desc schemas/event.proto
```

Инструмент визуализации

Роль `bi` — ось `bi`. Веб-интерфейс — `http://localhost:8088`. Источник данных дашборда — аналитическое хранилище: хост `warehouse`, порт 8123, пользователь `student`, пароль `student`, база `mart`.

Инструмент	Вход	Подключение к хранилищу
Apache Superset	`student` / `student`	база данных ClickHouse Connect
DataLens	`admin` / `student`	подключение ClickHouse в рабочей книге, TLS выключен
Metabase	учётная запись создаётся при первом входе	база данных ClickHouse

Таблица 5.5: Инструменты визуализации.

Строка подключения Superset — `clickhousedb://student:student@warehouse:8123/mart`. DataLens отвечает по HTTP, поэтому TLS выключается в дополнительных настройках подключения. DataLens — девять контейнеров: интерфейс, сервисы данных, своя база и Temporal; первый запуск занимает несколько минут.

Правила оформления графиков, дополняющие лекцию 5:

- **Цвет несёт смысл, а не украшает.** Одна категория — один цвет во всех графиках дашборда; для величины — последовательная шкала одного тона, для отклонения от нуля — расходящаяся. Больше семи категорий цветом не различаются: остальное сворачивается в «прочее» или получает подписи.
- **Доступность.** Пары красный — зелёный не несут различия одни: добавляются подписи, форма маркера или яркость. Текст на графике читается при печати в градациях серого.
- **Агрегация названа.** Подпись оси или заголовок говорит, что посчитано и за какой интервал: «правок в час», а не «правки». Среднее рядом с медианой, если распределение с длинным хвостом.
- **Детализация по шагам.** Дашборд открывается общим видом, детали — по фильтру или переходу на следующий график, а не сотней точек на одном.
- **Время — по горизонтали, в одном часовом поясе.** Часовой пояс назван; пропуски в данных видны как пропуски, а не соединяются линией.

Каталог данных

Каталог данных — файл `CATALOG.md` рабочего репозитория. В нём описан каждый набор данных платформы: что это, где лежит, откуда берётся и кто его читает. Каталог ведётся с работы 1: каждая работа, создающая новый набор, добавляет в него строку, а приёмка работы проверяет её. Описание сверяется со стендом: строка, противоречащая тому, что лежит в хранилище, считается ошибкой.

Поля строки

Поле	Что записывается
Имя	у файлов — `<слой>.<набор>`, то же имя стоит в пути; у таблиц и тем — полное имя: `lake.clean.events`, `mart.fact_events`
Архитектура и слой	файлы озера, таблица lakehouse или витрина хранилища; слой внутри архитектуры
Где лежит	путь в хранилище, имя таблицы или темы брокера
Формат	формат хранения и кодек сжатия
Партиционирование и сортировка	ключи партиций с единицей времени и порядок строк внутри файла или таблицы
Источник	из какого набора и каким заданием получен; для слоя сырых данных — источник и режим сборщика
Потребители	наборы, задания и дашборды, которые его читают; «не используется с ЛРN» — если потребителей не осталось
Свежесть	как часто обновляется и на сколько отстаёт от событий источника
Рост в 1000 раз	что станет узким местом при росте объёма на три порядка и что придётся изменить

Таблица 6.1: Поля строки каталога данных.

Поле «Рост в 1000 раз» — оценка, а не пересказ лекции. Она опирается на замеры своего набора: объём в сутки, число и размер файлов в партиции, время запроса замеров. По ним видно, что упрётся первым — число файлов, размер партиции, листинг, память движка или время загрузки — и какое решение сдвигает предел.

Набор не удаляется из каталога, когда перестаёт использоваться: строка остаётся с пометкой в поле «Потребители». Так видно, откуда взялись наборы, построенные поверх него.

Форма

Строка — раздел файла со списком полей: таблица на девять колонок в Markdown не читается.

```
## clean.events

- Архитектура и слой: озеро данных, слой очищенных данных
- Где лежит: s3://lake/clean/events/
- Формат: Parquet, zstd
- Партиционирование и сортировка: event_date=ГГГГ-ММ-ДД по времени события UTC;
  внутри файла — по event_time
- Источник: raw.stream и raw.history, задание jobs/clean.sql
- Потребители: таблица lakehouse lake.clean.events (ЛР4)
- Свежесть: раз в сутки, отставание до 26 ч
- Рост в 1000 раз: 90 ГБ в сутки, партиция в сотни файлов по 64 МБ;
  узкое место — листинг и число файлов, нужна партиция по часу и компакция
```

Строка каталога данных.

Соглашения озера

Перед строками наборов в `CATALOG.md` записываются соглашения, которые действуют для всех наборов. Их вводит работа 3, дальше они только соблюдаются:

- слои и их префиксы в bucket'е, назначение каждого слоя;
- шаблон ключа объекта: слой, набор, партиции, имя файла;
- имена и формат ключей партиций, единица времени и часовой пояс;
- правило имени файла, по которому повторная запись заменяет файл, а не добавляет копию;
- срок хранения каждого слоя и порядок удаления устаревшего.

Соглашение проверяется на стенде: листинг bucket'а должен совпадать с шаблоном без исключений.

Связи сервисов

Раздел «Связи сервисов» в `CATALOG.md` описывает, кто к кому обращается на стенде своего варианта. Его заводит работа 4; каждая следующая работа, запускающая новый сервис, добавляет связи этого сервиса. Связь — подраздел со списком полей, как строка набора:

Поле	Что записывается
Клиент → сервис	кто обращается и к кому: роль и продукт варианта; для заданий и программ — имя файла
Адрес	адрес, по которому обращается клиент
Протокол	REST-каталог Iceberg, API S3, SQL по HTTP или протоколу MySQL, протокол брокера
Учётные данные	каким ключом или учётной записью входит клиент; «без входа», если сервис вход не требует
Где задано	файл подключения, переменные `.env`, код задания или команда SQL

Таблица 6.2: Поля связи сервисов.

Путь запроса движка к данным — две связи: движок → каталог таблиц (метаданные и расположение файлов таблицы) и движок → объектное хранилище (сами файлы, своим ключом). Адрес записывается тот, по которому обращается клиент: из контейнера — имя роли, со стенда вне контейнеров — `localhost`. `localhost` внутри контейнера — сам контейнер, а не стенд.